SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

FACULTY WORKLOAD AND TIMETABLE MANAGEMENT SYSTEM

Item	Details
Program	B.Tech CSE (Artificial Intelligence and Machine Learning)
Section	AIML Leopards
Faculty	C Nagaraju
Date of Submission	12/10/2026
Academic Year	2026–27

Submitted by

Submitted by
Nakshatra Vijay Vargi | Roll Number: 25WU0102176

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course.

I thank my course faculty, C Nagaraju, for the guidance, suggestions and support throughout the design and development of this project. This guidance helped me apply ER modelling, relational schema design, normalization, SQL constraints, queries and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled “Faculty Workload and Timetable Management System”, submitted for the Database Management Systems Project-Based Learning activity, is my individual academic work carried out under the guidance of C Nagaraju. I acknowledge the tools and documentation used and accept responsibility for the accuracy of the submitted report, database design, source code and results.

Signature: _____

Date: 12/10/2026 Place: Hyderabad

Name: Nakshatra Vijay Vargi

Roll Number: 25WU0102176 Programme: B.Tech CSE (AI & ML)

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
Keywords	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	11
2. LOGICAL DATABASE DESIGN	13
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	13
2.3 Functional Dependencies	14
2.4 Normalization up to Third Normal Form (3NF)	14
2.5 Data Dictionary	14
2.6 Implementation of Logical Design: DDL and Sample Data	15
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	18
3.1 Application Overview	18
3.2 Development Environment and Database Connectivity	18
3.3 Application Modules and Database Mapping	19
3.4 CRUD Operations	19
3.5 Business Rules, Validation and Error Handling	19
3.6 Reports and Database Operations	20
3.7 Application Screens	20
3.8 Testing and Results	21
4. CONCLUSION	23
4.1 Limitations and Future Enhancements	23
5. REFERENCES	25
6. APPENDICES	26

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	9
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	10
Table 4	Entity and Relationship Summary	11
Table 5	Relational Schema	13
Table 6	Keys and Integrity Constraints	13
Table 7	Normalization Status up to 3NF	14
Table 8	Data Dictionary	14
Table 9	Database Implementation Environment	15
Table 10	Application Development Environment	18
Table 11	Application Module-to-Database Mapping	19
Table 12	CRUD Operations	19
Table 13	Business Rules, Validation and Error Handling	19
Table 14	Reports and Database Operations	20
Table 15	Test Cases	21

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Core Relational ER Diagram	11
Figure 2	Application Architecture	18
Figure 3	Faculty Overview and Workload Display	21
Figure 4	SQLite Faculty Table Query Output	21

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FK	Foreign Key
HTML	HyperText Markup Language
PK	Primary Key
SQL	Structured Query Language
UI	User Interface

ABSTRACT

Faculty workload and timetable planning requires accurate management of faculty profiles, course information, teaching assignments, rooms and class time slots. When these records are maintained separately, changes can be inconsistent and timetable clashes can be difficult to detect. This project develops a database-backed Faculty Workload and Timetable Management System using Python, Flask and SQLite. The design separates faculty, courses, rooms, time slots, assignments, timetable entries and substitutions into related tables linked by primary and foreign keys. The application provides screens for faculty and course records, room management, faculty-course-section assignments, timetable scheduling and substitution records. Workload summaries are calculated from assignment hours, while timetable scheduling checks for room and faculty clashes in the selected slot before saving a class. The SQLite database provides persistent local storage, and SQL joins and aggregate queries support workload and timetable views. The prototype demonstrates the practical use of relational modelling, constraints, queries and application-level validation in academic scheduling. Automated timetable optimization, authentication and production deployment remain possible future improvements.

KEYWORDS

Keywords: Faculty workload, timetable management, SQLite, Flask, relational database, SQL

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Faculty workload and timetable coordination requires accurate records of faculty members, courses, sections, rooms and time slots. Managing these details in separate spreadsheets or disconnected records can cause duplication, inconsistent updates and difficulty in tracking teaching hours. Timetable preparation must also ensure that a faculty member or room is not allocated to two classes at the same time. The project addresses these issues through a relational database and a Python web application that centralizes the records and supports workload visibility and timetable clash checks.

1.2 Objectives

- Design a relational schema for faculty, courses, rooms, time slots, assignments, timetable entries and substitutions.
- Develop a Flask application that reads and writes persistent data in SQLite.
- Calculate faculty workload from weekly hours recorded in assignment records.
- Validate timetable entries against room and faculty clashes in the selected slot.

1.3 Scope and Assumptions

Table 1. Scope and Assumptions

Item	Description
In Scope	Faculty, courses, rooms, assignments, workload overview, timetable scheduling, clash checks and substitution records.
Out of Scope / Exclusions	Automatic timetable optimization, authentication, multi-user hosting, institutional integration and notification delivery.
Assumptions	The prototype runs locally; sample data is illustrative; each timetable entry references an assignment, room and time slot.

1.4 Proposed Solution Strategy

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Academic administrator or demonstrator managing local scheduling records.
Major Application Modules	Dashboard, faculty, courses, rooms, assignments, timetable and substitutions.
Proposed Workflow	Maintain faculty/course/room data; create assignment; choose room and time slot; validate and save timetable entry; review workload.
Important Business Rules	Prevent duplicate faculty-course-section assignments; reject assignments exceeding a faculty workload limit; check room/faculty clashes in the selected slot.
Expected Reports / Outputs	Faculty workload overview, course/assignment lists, timetable listing, clash messages and substitution register.

1.5 Functional Requirements

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Create and display faculty records.	High
FR-02	Create and display course and room records.	High
FR-03	Assign faculty to a course and section with weekly hours.	High
FR-04	Schedule an assignment in a room and time slot.	High
FR-05	Check room and faculty clashes before saving a timetable entry.	High

1.6 Entity-Relationship Diagram (ERD)

The diagram below summarizes the core relations implemented in the current SQLite application. Verify the final schema before submission.

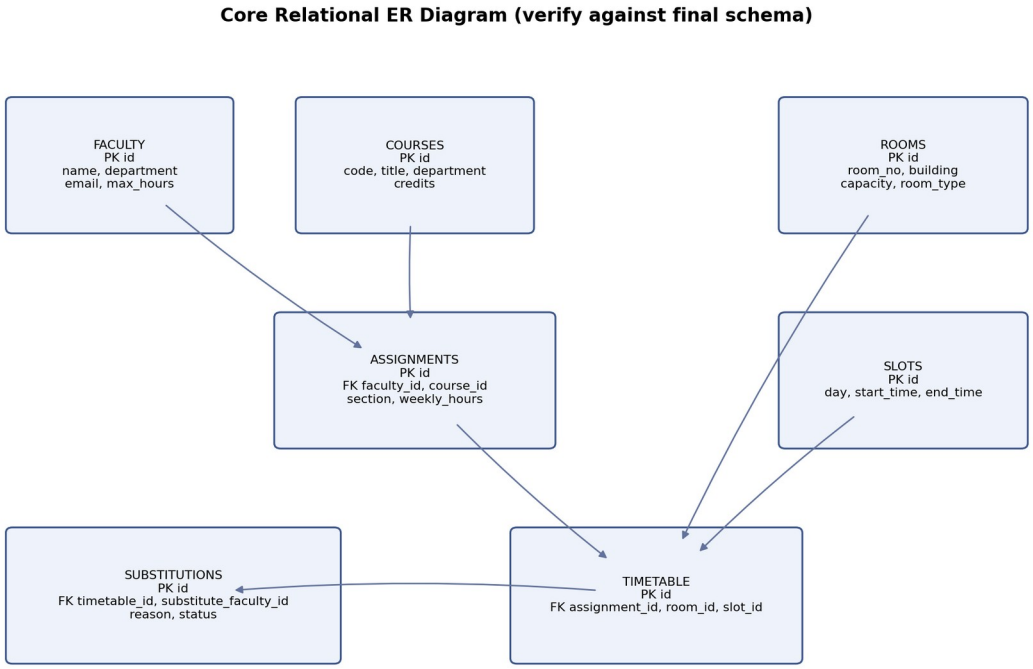


Figure 1. Core Relational ER Diagram

Figure 1. Core Relational ER Diagram (verify against final schema)

1.7 Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Faculty	id PK; name, department, email, max_hours	Stores faculty identity and workload limit.
Courses	id PK; code, title, department, credits	Stores course catalogue.
Rooms	id PK; room_no, building, capacity, room_type	Stores class locations.
Slots	id PK; day, start_time, end_time	Stores available class time slots.
Assignments	id PK; faculty_id FK, course_id FK, section, weekly_hours	Associates faculty with course and section.
Timetable	id PK; assignment_id FK, room_id FK, slot_id FK	Schedules an assignment into a room and slot.
Substitutions	id PK; timetable_id FK, substitute_faculty_id FK, reason, status	Records replacement faculty for a scheduled class.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	faculty	Faculty profile and weekly teaching-hour limit.
2	courses	Course code, title, department and credits.
3	rooms	Room number, building, capacity and type.
4	slots	Day and start/end times.
5	assignments	Faculty-course-section relationship and weekly hours.
6	timetable	Links an assignment to a room and slot.
7	substitutions	Replacement faculty, reason and status.

2.2 Keys and Integrity Constraints

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
slots	PRIMARY KEY	id	Unique faculty identifier.
	NOT NULL / DEFAULT	name, department, max_hours	Requires key fields; max_hours defaults to 16.
	PRIMARY KEY / UNIQUE	id, code	Course code is unique.
	PRIMARY KEY / UNIQUE	id, room_no	Room number is unique.
	PRIMARY KEY / UNIQUE	id; day, start_time, end_time	Prevents duplicate slot tuples.
Application logic	FOREIGN KEY	faculty_id, course_id	References existing faculty and courses.
	NOT NULL / DEFAULT	section, weekly_hours	Requires section and weekly-hours value; weekly_hours defaults to 3.
	FOREIGN KEY / UNIQUE	assignment_id, room_id, slot_id; room_id + slot_id	References valid records and prevents duplicate room-slot pairs.
	FOREIGN KEY / NOT NULL / DEFAULT	timetable_id, substitute_faculty_id, reason, status	References scheduled class and substitute; status defaults to Approved.
	Validation	assignment tuple; faculty hours; slot clash	Rejects duplicate assignments, excessive workload and room/faculty clashes.

2.3 Functional Dependencies

The implemented schema supports the following key functional dependencies: faculty.id → name, department, email, max_hours; courses.id → code, title, department, credits and courses.code → id, title, department, credits; rooms.id → room_no, building, capacity, room_type and rooms.room_no → building, capacity, room_type; slots.id → day, start_time, end_time; assignments.id → faculty_id, course_id, section, weekly_hours; timetable.id → assignment_id, room_id, slot_id; substitutions.id → timetable_id, substitute_faculty_id, reason, status.

2.4 Normalization

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
faculty	Yes*	Yes*	Yes*	Yes*	Single-column PK; descriptive attributes depend on id.
courses	Yes*	Yes*	Yes*	Yes*	id determines details; code is unique.
rooms	Yes*	Yes*	Yes*	Yes*	id determines details; room_no is unique.
slots	Yes*	Yes*	Yes*	Yes*	id determines day and time values.
assignments	Yes*	Yes*	Yes*	Yes*	Assignment details stored separately from faculty/course descriptions.
timetable	Yes*	Yes*	Yes*	Yes*	Assignment, room and slot are referenced by foreign keys.
substitutions	Yes*	Yes*	Yes*	Yes*	Substitution details stored separately from class/faculty details.

2.5 Data Dictionary

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
faculty	id	INTEGER	PK	Unique faculty identifier.
faculty	name	TEXT	NOT NULL	Faculty name.
faculty	department	TEXT	NOT NULL	Department label.
faculty	email	TEXT	Optional	Faculty email address.
faculty	max_hours	INTEGER	NOT NULL DEFAULT 16	Maximum weekly teaching hours.
courses	id	INTEGER	PK	Unique course identifier.
courses	code	TEXT	UNIQUE, NOT NULL	Course code.

Table	Attribute	Data Type	Key / Constraint	Description
courses	title	TEXT	NOT NULL	Course title.
courses	department	TEXT	NOT NULL	Owning department.
courses	credits	INTEGER	NOT NULL DEFAULT 3	Course credit value.
rooms	id	INTEGER	PK	Unique room identifier.
rooms	room_no	TEXT	UNIQUE, NOT NULL	Room label.
rooms	building	TEXT	NOT NULL	Building or block.
rooms	capacity	INTEGER	NOT NULL DEFAULT 40	Room capacity.
rooms	room_type	TEXT	NOT NULL DEFAULT 'Classroom'	Room/lab type.
slots	id	INTEGER	PK	Unique time-slot identifier.
slots	day	TEXT	NOT NULL	Day of week.
slots	start_time	TEXT	NOT NULL	Slot start time.
slots	end_time	TEXT	NOT NULL	Slot end time.
assignments	id	INTEGER	PK	Unique assignment identifier.
assignments	faculty_id	INTEGER	FK → faculty.id	Assigned faculty.
assignments	course_id	INTEGER	FK → courses.id	Assigned course.
assignments	section	TEXT	NOT NULL	Class section.
assignments	weekly_hours	INTEGER	NOT NULL DEFAULT 3	Weekly teaching hours.
timetable	id	INTEGER	PK	Unique timetable entry.
timetable	assignment_id	INTEGER	FK → assignments.id	Assignment being scheduled.
timetable	room_id	INTEGER	FK → rooms.id	Room used.
timetable	slot_id	INTEGER	FK → slots.id	Time slot used.
substitutions	id	INTEGER	PK	Unique substitution record.
substitutions	timetable_id	INTEGER	FK → timetable.id	Class requiring substitution.
substitutions	substitute_faculty_id	INTEGER	FK → faculty.id	Replacement faculty.
substitutions	reason	TEXT	NOT NULL	Reason for substitution.
substitutions	status	TEXT	NOT NULL DEFAULT 'Approved'	Substitution status.

2.6 Implementation of Logical Design: DDL and Sample Data

Table 9. Database Implementation Environment

The core database uses seven tables: faculty, courses, rooms, slots, assignments, timetable and substitutions. The following statements are representative extracts from the application's SQLite schema.

```
CREATE TABLE faculty (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL, department TEXT NOT NULL, email TEXT,
  max_hours INTEGER NOT NULL DEFAULT 16
);

CREATE TABLE assignments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  faculty_id INTEGER NOT NULL, course_id INTEGER NOT NULL,
  section TEXT NOT NULL, weekly_hours INTEGER NOT NULL DEFAULT 3,
  FOREIGN KEY (faculty_id) REFERENCES faculty(id),
  FOREIGN KEY (course_id) REFERENCES courses(id)
```

```
);

CREATE TABLE timetable (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
   assignment_id INTEGER NOT NULL, room_id INTEGER NOT NULL, slot_id INTEGER NOT NULL,
  FOREIGN KEY (assignment_id) REFERENCES assignments(id) ON DELETE CASCADE,
  FOREIGN KEY (room_id) REFERENCES rooms(id),
  FOREIGN KEY (slot_id) REFERENCES slots(id),
  UNIQUE (room_id, slot_id)
);

INSERT INTO faculty (name, department, email, max_hours)
VALUES ('Dr. Ananya Rao', 'Computer Science', 'ananya.rao@woxsen.edu.in', 16);
```

The application seeds demonstration data only when the relevant tables are empty. The complete schema and seed-data logic are contained in the submitted application file app.py; include app.py or a separate SQL export in Appendix A if required by the course.

Item	Details
DBMS	SQLite
DBMS Version	Exact runtime version not recorded; SQLite version is provided by the installed Python environment.
Number of Tables	7 core tables.

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

3.2 Development Environment and Database Connectivity

The application uses Flask route handlers to validate form submissions and execute parameterized SQL statements through Python's sqlite3 connector. The database file is resolved relative to the application source file, and foreign-key enforcement is enabled for each connection.

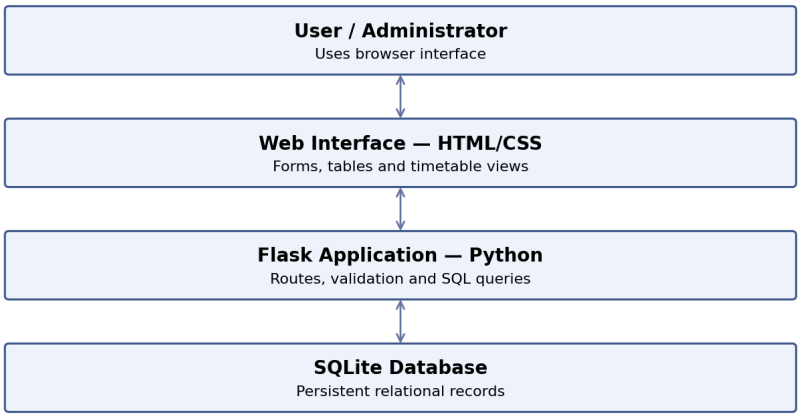


Figure 2. Application Architecture

Table 10. Application Development Environment

Item	Details
Programming Language	[Java / Python]
DBMS	[MySQL / PostgreSQL / SQLite]
IDE / Framework / GUI Toolkit	[Tool names]
Database Connector / Driver	[Connector / driver used]

Operating Environment	[OS / other details]
Repository Link	[GitHub / repository link]

3.3 Application Modules and Database Mapping

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Summary counts, timetable and workload overview	faculty, courses, rooms, timetable, assignments, slots	Read / aggregate
Faculty	Add and list faculty; show assigned hours	faculty, assignments	Create, read
Courses	Add and list courses	courses	Create, read
Rooms	Add and list rooms	rooms	Create, read
Assignments	Link faculty, course and section; check workload and duplicates	assignments, faculty, courses	Create, read, validate
Timetable	Schedule class and check room/faculty clashes	timetable, assignments, rooms, slots, faculty	Create, read, conflict check
Substitutions	Record substitute faculty and reason	substitutions, timetable, faculty	Create, read

3.4 CRUD Operations

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Add faculty, course, room, assignment, timetable entry or substitution	faculty, courses, rooms, assignments, timetable, substitutions	Creation forms and success/error messages.
Read / Search	View dashboard, faculty, course, room, assignment and timetable records	All core tables through SELECT/JOIN	Visible lists and workload summary.
Update	No general edit/update form confirmed in current version	Not confirmed	Do not claim update support.
Delete / Controlled Removal	Remove a timetable entry	timetable	Delete route exists; verify through UI.

3.5 Business Rules, Validation and Error Handling

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Foreign-key integrity	Foreign keys enabled on each SQLite connection and declared in schema.	Invalid references should be rejected.
Duplicate assignment	Checks existing faculty/course/section before insert.	Displays duplicate-assignment error.
Workload limit	Checks current assigned hours plus proposed hours against max_hours.	Rejects assignment if limit is exceeded.
Room/faculty clash	Checks existing timetable entry with same slot and same room or faculty.	Displays conflict message before insert.
Unique room-slot	Database UNIQUE(room_id, slot_id) constraint.	Prevents a duplicate room/slot pair.

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Faculty workload	LEFT JOIN, SUM, COALESCE, GROUP BY	Assigned hours compared with max_hours.
Timetable view	JOIN across timetable, slots, assignments, faculty, courses and rooms	Scheduled classes ordered by day and time.
Dashboard totals	COUNT(*) across faculty, courses, rooms and timetable	Summary cards.
Assignment display	JOIN assignments to faculty and courses	Faculty-course-section register.
Conflict detection	SELECT for same slot and room or faculty	Prevents a conflicting class from being saved.
Substitution list	JOIN substitutions to timetable, slot, assignment, course and faculty	Replacement faculty and reason.

3.7 Application Screens

The Faculty Overview screen displays faculty names, departments, email addresses and assigned workload against each faculty member's maximum weekly hours.

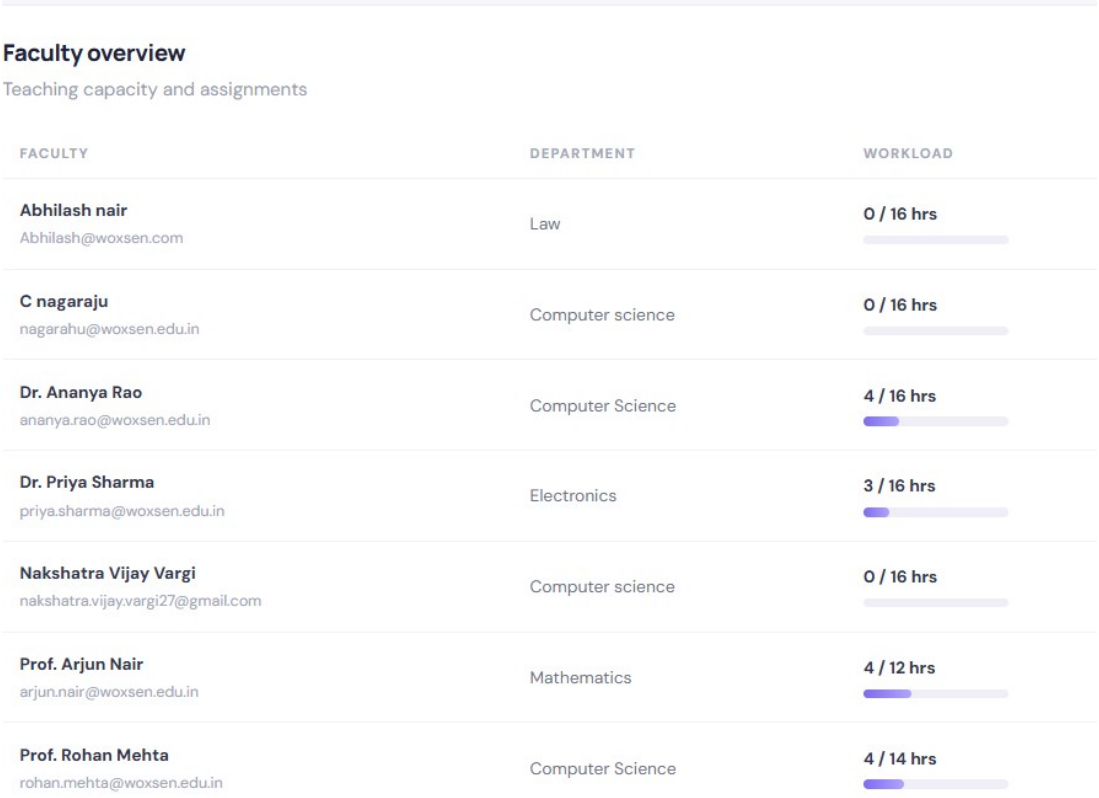
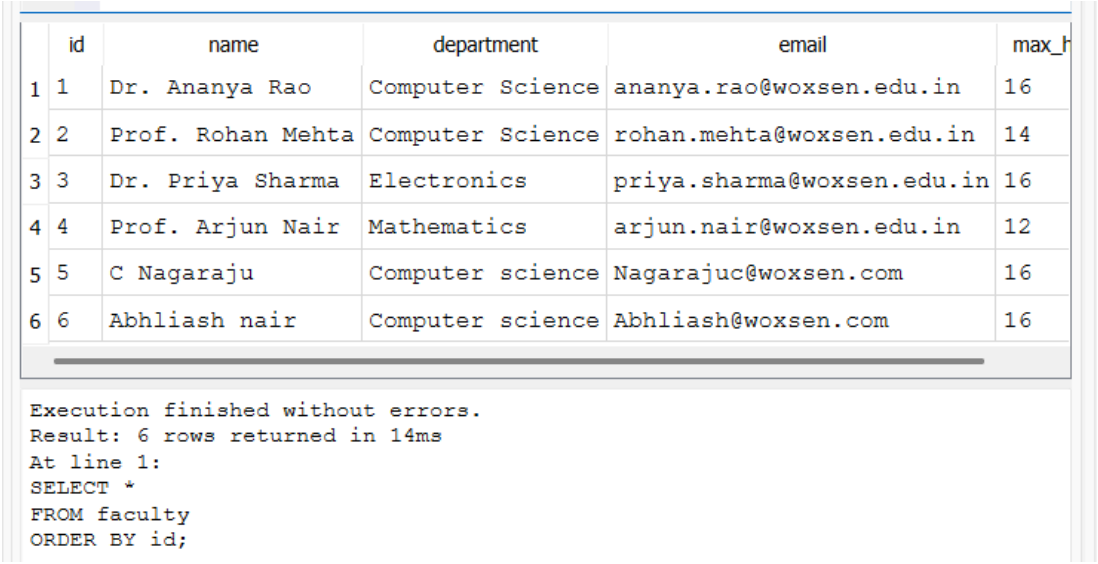


Figure 3. Faculty Overview and Workload Display

The database was also inspected using DB Browser for SQLite. The screenshot shows a SELECT query against the faculty table, with six rows returned at the time of capture.



	id	name	department	email	max_h
1	1	Dr. Ananya Rao	Computer Science	ananya.rao@woxsen.edu.in	16
2	2	Prof. Rohan Mehta	Computer Science	rohan.mehta@woxsen.edu.in	14
3	3	Dr. Priya Sharma	Electronics	priya.sharma@woxsen.edu.in	16
4	4	Prof. Arjun Nair	Mathematics	arjun.nair@woxsen.edu.in	12
5	5	C Nagaraju	Computer science	Nagarajuc@woxsen.com	16
6	6	Abhliash nair	Computer science	Abhliash@woxsen.com	16

```
Execution finished without errors.
Result: 6 rows returned in 14ms
At line 1:
SELECT *
FROM faculty
ORDER BY id;
```

Figure 4. SQLite Faculty Table Query Output

Figure 4. SQLite Faculty Table Query Output

Figure 3. Faculty Overview and Workload Display

3.8 Testing and Results

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Database connection	Start app and open dashboard	Application reads SQLite records	User reported the fixed app works; recheck final folder.	Reported working
TC-02	Faculty display	Open Faculty Overview	Faculty and workload values display	Screenshot shows faculty names, departments and hours.	Observed
TC-03	Faculty query	Run SELECT * FROM faculty ORDER BY id	Rows return without SQL error	Six rows were returned in supplied query screenshot at that time.	Observed
TC-04	Assignment creation	Submit a valid faculty/course/section assignment	Assignment appears in assignment list	Retest final version.	To verify
TC-05	Workload limit	Submit assignment exceeding max_hours	Application rejects assignment with message	Retest final version.	To verify
TC-06	Room conflict	Schedule another class in occupied room and slot	Conflict message; no new entry	Retest final version.	To verify
TC-07	Faculty conflict	Schedule same faculty in same slot using another room	Conflict message; no new entry	Retest final version.	To verify

Observed development evidence confirms that the SQLite database opens in DB Browser and that a SELECT query against the faculty table returns records without errors. The Faculty Overview screen displays faculty names, departments and workload values. The remaining negative test cases are marked “To verify” until they are rerun against the final application.

CHAPTER 4

CONCLUSION

The Faculty Workload and Timetable Management System demonstrates how a relational database can support academic scheduling. The prototype separates faculty, courses, rooms, slots, assignments, timetable entries and substitutions, and links related records through primary and foreign keys. The Flask interface supports data-entry and viewing workflows, while SQL joins and aggregate queries provide workload and timetable views. The project provides practical experience in schema design, database connectivity, constraints, SQL queries and application-level validation.

The Faculty Workload and Timetable Management System demonstrates how a relational database can support academic scheduling. The prototype separates faculty, courses, rooms, time slots, assignments, timetable entries and substitutions, and connects related records through primary and foreign keys. A Python Flask interface provides the main viewing and data-entry workflows, while SQL joins and aggregate queries support workload summaries and timetable displays. The project provided practical experience in schema design, normalization, database connectivity, SQL queries and application-level validation.

4.1 Limitations and Future Enhancements

The current version is a local prototype and does not provide automated timetable optimization or user authentication. The timetable conflict check compares exact slot identifiers rather than arbitrary overlapping time intervals. Future work could add these features, improve reports, and add timetable exports and substitution notifications.

- Add automated timetable generation with constraints for faculty workload, room capacity and section availability.
- Add authentication and role-based permissions before deployment beyond a local demonstration.
- Expand reporting and test overlapping time intervals, exports and substitution notifications.

REFERENCES

[1] [1] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: Oct. 12, 2026.

[2] [2] Pallets Projects, “Flask Documentation.” [Online]. Available: https://flask.palletsprojects.com/. Accessed: Oct. 12, 2026.

[3] [3] SQLite Consortium, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html. Accessed: Oct. 12, 2026.

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

Representative SQL (confirm against final app schema):

```
CREATE TABLE faculty (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, department TEXT NOT NULL, email TEXT, max_hours INTEGER NOT NULL DEFAULT 16);
```

APPENDIX B - APPLICATION SOURCE CODE

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Appendix C is reserved for additional screenshots and test outputs that are actually captured from the final application.

APPENDIX D - SQL QUERY CATALOGUE

This appendix documents representative SQL operations used by the application. The examples illustrate the database concepts behind the dashboard and management screens. They are intended to be read alongside the actual app.py source file; table and column names should match the deployed database schema.

D.1 Faculty workload summary

A LEFT JOIN ensures that faculty members with no assignments still appear in the overview. COALESCE converts a missing SUM result into zero.

```
SELECT f.name, f.department, f.max_hours,
       COALESCE(SUM(a.weekly_hours), 0) AS assigned_hours
FROM faculty AS f
LEFT JOIN assignments AS a ON a.faculty_id = f.id
GROUP BY f.id
ORDER BY assigned_hours DESC;
```

D.2 Timetable listing

The timetable query joins the scheduled entry to its slot, assignment, faculty, course and room. This avoids storing repeated names in the timetable table and keeps the database normalized.

```
SELECT s.day, s.start_time, s.end_time, c.code,
       c.title, a.section, f.name AS faculty, r.room_no
FROM timetable t
JOIN slots s ON s.id = t.slot_id
JOIN assignments a ON a.id = t.assignment_id
JOIN faculty f ON f.id = a.faculty_id
JOIN courses c ON c.id = a.course_id
JOIN rooms r ON r.id = t.room_id
ORDER BY s.day, s.start_time;
```

D.3 Conflict detection logic

Before inserting a timetable entry, the application searches the selected slot for either the same room or the same assigned faculty member. If a match is found, it displays a conflict message and does not insert the new entry. The current implementation compares slot IDs, so arbitrary overlapping time ranges are not detected unless they are represented by the same slot record.

```
SELECT t.id
FROM timetable t
JOIN assignments a ON a.id = t.assignment_id
WHERE t.slot_id = ?
  AND (t.room_id = ? OR a.faculty_id = ?)
LIMIT 1;
```

Parameterized placeholders (?) are used for user-supplied values. Parameterized SQL separates data from query syntax and helps prevent SQL injection.

APPENDIX E - APPLICATION USER GUIDE

The web application is a local Flask prototype. Start it from the project directory using the Python environment in which Flask is installed. The database file is resolved relative to app.py, so the app should be run from the extracted project folder rather than copied away from its database.

E.1 Start the application

- Open the project folder in a terminal.
- Activate the project's virtual environment if one is being used.
- Run: python app.py
- Open the local address printed by Flask in the terminal, usually http://127.0.0.1:5000.
- Keep the terminal running while using the application. Stop the development server with Ctrl+C.

E.2 Recommended workflow

Step	Screen	Action and expected outcome
1	Faculty	Review existing faculty. Add a faculty name, department, optional email and maximum weekly teaching hours.
2	Courses	Add a unique course code, title, department and credit value.
3	Rooms	Add a unique room number, building, capacity and room type.
4	Assignments	Select faculty, course and section, then enter weekly hours. The app checks duplicate assignments and workload limits.
5	Timetable	Choose an assignment, room and time slot. The app checks the selected slot for room/faculty conflicts before saving.
6	Substitutions	Select a scheduled class, substitute faculty and reason, then submit the record.

E.3 Data integrity reminders

- Use a unique course code and room number; duplicates are rejected by the database constraints.
- Confirm that weekly assignment hours are realistic and do not exceed the faculty member's configured maximum.
- Check the timetable after each change. A saved substitution is a separate record and does not replace the original timetable assignment.
- The prototype has no confirmed authentication or role-based access control; do not expose it publicly without additional security work.

APPENDIX F - DETAILED TEST PLAN

The table distinguishes evidence already observed in the supplied screenshots from tests that still need to be executed on the final project folder. A test should only be marked Passed after the action has been performed and the actual result recorded.

ID	Test scenario	Procedure	Expected result	Status
F-01	Database opens	Start app and open dashboard.	Dashboard loads records from the local SQLite file.	Previously reported working; recheck final copy
F-02	Faculty listing	Open Faculty Overview.	Faculty, department and workload values are visible.	Observed in supplied screenshot
F-03	Read query	Run SELECT * FROM faculty ORDER BY id in DB Browser.	Rows are returned without SQL error.	Observed; six rows in that capture
F-04	Duplicate course code	Add a course with an existing code.	Insert is rejected and an error message is shown.	To verify
F-05	Duplicate room number	Add a room number already present.	Insert is rejected and an error message is shown.	To verify
F-06	Duplicate assignment	Submit the same faculty/course/section combination twice.	Second assignment is not inserted.	To verify
F-07	Workload limit	Try an assignment that exceeds max_hours.	Assignment is rejected with a workload warning.	To verify
F-08	Room clash	Schedule a second class in the same room and slot.	Conflict is reported; no new timetable row is inserted.	To verify
F-09	Faculty clash	Schedule the same faculty in the same slot using another room.	Conflict is reported; no new timetable row is inserted.	To verify
F-10	Valid substitution	Select an existing class and another faculty member.	Substitution record appears in the list.	To verify
F-11	Delete timetable entry	Use the remove-class action on an existing entry.	Selected timetable row is removed.	To verify

For repeatable testing, use a copy of the database or a fresh test database. Record the date, database file used, test input, observed message and final row count for each negative test. Avoid using the only working demonstration database for destructive experiments.

APPENDIX G - IMPLEMENTATION NOTES AND REPRODUCIBILITY

G.1 Project components

Component	Purpose	Verification note
app.py	Flask routes, schema initialization, seed data, SQL queries and validation.	Source file is the authoritative implementation reference.
faculty_timetable.db	Persistent SQLite data store.	Open with DB Browser for SQLite; confirm the exact file path.
templates/	HTML templates for dashboard and module screens.	Inspect the project folder to confirm all referenced templates are present.
Python sqlite3	Database driver included with Python.	Connections enable PRAGMA foreign_keys = ON.
Flask	Web framework and routing.	Run locally for demonstration; debug mode is enabled in the current script.

G.2 Reproducible setup checklist

- Keep app.py, the templates directory and faculty_timetable.db together in the project folder.
- Install Flask in the selected Python environment if it is not already installed.
- Back up the database before manually changing rows in DB Browser for SQLite.
- Run the application and confirm that the dashboard, faculty, courses, rooms, assignments, timetable and substitutions routes open.
- Capture new screenshots from the final folder for any test results that will be claimed in the submitted report.

G.3 Known limitations

The current implementation is a demonstration prototype rather than a production scheduling system. It has no confirmed login system, audit log, backup automation or concurrent multi-user deployment setup. The timetable clash check uses slot identity rather than a general interval-overlap algorithm. Error handling in several create routes uses broad exception handling, which is simple for a prototype but should be refined for deployment. These limitations are recorded to distinguish implemented behaviour from future enhancements.

G.4 Suggested next development steps

Prioritize automated tests for duplicate records, workload limits and timetable clashes; add database backup/export; introduce server-side validation with specific error messages; implement user authentication and roles; then consider automatic timetable generation using constraints for faculty, room, section and time availability.